



**UNIVERSIDAD
DE GRANADA**

Práctica 4

Árboles Binarios de Búsqueda para la gestión de eventos

Estructura de Datos

Curso 2025/2026

Departamento de Ciencias de la Computación e I.A.
Universidad de Granada

18 de diciembre de 2025

Índice

1. Introducción	1
2. Objetivos y requisitos	1
3. Documentación	1
3.1. Definición del T.D.A.	2
3.2. Operaciones	2
3.3. Implementación del T.D.A.	2
4. Ejercicio	2
4.1. Módulos a desarrollar	3
4.2. Compilación y Makefile	3
5. Entrega de la práctica	3
5.1. Estructura de carpetas	4
6. Rúbrica de evaluación	4

1 Introducción

Los árboles binarios de búsqueda constituyen una de las estructuras de datos fundamentales en informática para organizar información de forma eficiente cuando es necesario mantenerla ordenada. A diferencia de las estructuras lineales, los árboles permiten insertar, buscar y recorrer los datos manteniendo un orden implícito, sin necesidad de aplicar algoritmos de ordenación adicionales.

En esta práctica se propone desarrollar un sistema sencillo de gestión de eventos utilizando un **árbol binario de búsqueda (ABB)**. Cada evento estará definido por una fecha y una descripción asociada, y se almacenará en el árbol de manera que los eventos queden organizados cronológicamente.

El objetivo principal es que el estudiante implemente manualmente la estructura del árbol, gestionando punteros y memoria dinámica, y aplicando técnicas de programación recursiva propias de las estructuras jerárquicas.

2 Objetivos y requisitos

Los objetivos de esta práctica son los siguientes:

1. Comprender el funcionamiento de los árboles binarios de búsqueda.
2. Diseñar e implementar un tipo de dato abstracto basado en árboles binarios.
3. Utilizar recorridos en árboles para mostrar información ordenada.
4. Aplicar recursividad y memoria dinámica en C++.
5. Documentar correctamente un T.D.A. siguiendo las convenciones de la asignatura.

Los requisitos previos para poder realizar esta práctica son:

1. Haber superado las prácticas anteriores.
2. Conocer el uso de punteros y memoria dinámica en C++.
3. Comprender el concepto de recursividad.
4. Conocer la definición teórica de los árboles binarios y sus recorridos básicos.

3 Documentación

El objetivo fundamental de un programador es que los tipos de datos abstractos (T.D.A.) que desarrolla puedan ser reutilizados por otros programadores sin necesidad de conocer los detalles de su implementación interna. Para ello, los módulos deben estar correctamente documentados, diferenciando claramente entre:

1. **Especificación:** describe la parte pública del T.D.A., indicando qué operaciones ofrece y cuál es su comportamiento.
2. **Implementación:** recoge la representación interna del T.D.A. y las decisiones de diseño adoptadas.

3.1 Definición del T.D.A.

En esta práctica se deberán definir dos tipos de datos abstractos: **Evento** y **ArbolEventos**. La definición de cada T.D.A. deberá realizarse en lenguaje natural, indicando claramente qué representa cada uno y qué información encapsula, sin hacer referencia a los detalles de implementación.

3.2 Operaciones

Todas las operaciones públicas de los T.D.A. deberán documentarse utilizando anotaciones `doxygen`, incluyendo al menos los campos `@brief`, `@param`, `@pre`, `@post` y `@return` cuando corresponda.

3.3 Implementación del T.D.A.

Para cada T.D.A. se deberá definir:

1. El **tipo de representación**.
2. El **invariante de representación**, que garantice la validez del estado interno.
3. La **función de abstracción**, que relacione la representación interna con el valor abstracto que se desea modelar.

4 Ejercicio

En esta práctica se deberá desarrollar un sistema de gestión de eventos utilizando un árbol binario de búsqueda. Cada evento estará definido por una fecha (día, mes y año) y una descripción textual.

Un evento se considerará anterior a otro si su fecha es cronológicamente menor. Este criterio deberá utilizarse para decidir la posición de inserción de los eventos en el árbol binario de búsqueda.

El sistema deberá permitir la carga de eventos desde un fichero de texto. Cada línea del fichero representará un evento independiente, que deberá ser procesado secuencialmente e insertado en el árbol binario de búsqueda según el criterio de orden establecido.

El formato del fichero de entrada será el siguiente:

```
día mes año descripción_del_evento
```

A continuación se muestra un ejemplo de fichero de texto válido (`eventos.txt`):

```
12 10 2025 Examen de Estructura de Datos
05 11 2025 Entrega practica de arboles binarios
21 09 2025 Inicio del curso academico
15 01 2026 Examen final del primer cuatrimestre
01 12 2025 Presentacion del proyecto final
30 09 2025 Primer control parcial
```

Los eventos del fichero no estarán necesariamente ordenados. El programa deberá leer el fichero línea a línea e insertar cada evento en el árbol, de forma que posteriormente puedan mostrarse en orden cronológico utilizando el iterador implementado.

Los pasos a realizar son los siguientes:

1. Diseñar un tipo de dato abstracto que represente un evento, definido por una fecha y una descripción.
2. Diseñar un segundo tipo de dato abstracto que almacene eventos utilizando un árbol binario de búsqueda, manteniendo el orden cronológico por fecha.
3. Implementar las operaciones básicas del árbol, incluyendo la inserción de eventos, la búsqueda por fecha y el cálculo de información estructural.
4. Implementar un **iterador que permita recorrer el árbol de eventos de forma ordenada**, respetando el criterio de orden establecido.
5. Utilizar dicho iterador para mostrar los eventos almacenados en orden cronológico.

Para cada T.D.A. el alumno deberá:

1. Definir el tipo en lenguaje natural.
2. Justificar la representación elegida.
3. Establecer el invariante de representación y la función de abstracción.
4. Implementar y documentar todas las operaciones necesarias.

4.1 Módulos a desarrollar

El proyecto deberá estructurarse en los siguientes módulos:

- `evento.h` y `evento.cpp`
- `arboleventos.h` y `arboleventos.cpp`
- `main.cpp`

4.2 Compilación y Makefile

El proyecto deberá incluir un archivo **Makefile** que permita:

- Compilar el programa principal mediante `make`.
- Eliminar archivos generados mediante `make clean`.
- Generar la documentación con `make doc`.

5 Entrega de la práctica

La práctica se entregará comprimida en un único fichero:

`apellidos_nombre_p4.zip`

No se admitirán prácticas que no respeten la estructura indicada o que incluyan ficheros objeto o ejecutables.

5.1 Estructura de carpetas

```
apellidos_nombre_p4/
|
|-- include/
|   |-- evento.h
|   |-- arboleventos.h
|
|-- src/
|   |-- evento.cpp
|   |-- arboleventos.cpp
|   |-- main.cpp
|
|-- build/
|-- datos/
|-- doc/
|-- Makefile
```

6 Rúbrica de evaluación

Criterio	Puntuación	Descripción
Implementación del T.D.A. Evento	2 puntos	- Definición correcta del evento. - Comparación adecuada por fecha.
Implementación del T.D.A. ArbolEventos	3 puntos	- Inserción correcta en el ABB. - Uso adecuado de recorridos y búsquedas. - Mantenimiento del invariante.
Recursividad y memoria dinámica	2 puntos	- Uso correcto de recursividad. - Gestión adecuada de memoria.
Documentación y abstracción	2 puntos	- Uso correcto de doxygen. - Invariante y función de abstracción bien definidos.
Estructura y entrega	1 punto	- Organización correcta del proyecto. - Makefile funcional.

Cuadro 1: Rúbrica de evaluación de la práctica